

Laporan Tugas Pemrograman Web

Jobsheet 5 dan 6



Oleh:

Muhammad Ramdhani Ariyadin (254107060018)

SIB 2-E

Link Github : <https://github.com/MuhammadRamdhaniAriyadin/Pemrograman-Web>

Jobsheet 5

1. Tambah validasi field baru — misalnya field ISBN di form Tambah Buku (yang saat ini tidak wajib diisi, ingat dari dokumentasi jobsheet-01 §4.4) validasi supaya hanya menerima angka dan tanda hubung.

Jawab

Menambahkan aturan regex `/^[0-9-]+$` pada form-tambah khusus untuk field isbn agar hanya menerima angka dan tanda hubung

```
// Tambahkan snippet
const isbn = form.querySelector("[name='isbn']");
if (isbn && isbn.value.trim() !== "") {
  const patternIsbn = /^[0-9-]+$/;
  if (!patternIsbn.test(isbn.value.trim())) {
    tampilkanError(isbn, "ISBN hanya boleh berisi angka dan tanda hubung (-).");
    valid = false;
  } else {
    hapusError(isbn);
  }
}
```

Uji coba



ISBN

asc

ISBN hanya boleh berisi angka dan tanda hubung (-)

Stok

2. Tambah animasi sederhana pada `initNavToggle` — misalnya tambahkan class CSS transition pada header nav di `style.css` supaya menu terbuka/tertutup dengan efek geser halus, alih-alih langsung muncul/hilang seketika.

Jawab

Mengganti `display: none` / `display: block` dengan manipulasi `max-height` dan `opacity` di CSS agar transisi buka/tutup menu menjadi halus.

```
/* mengganti display none block dengan transisi max-height dan opacity */
header nav {
  width: 100%;
  order: 3;
  margin-top: 0;
  max-height: 0;
  opacity: 0;
  overflow: hidden;
  transition: max-height 0.4s ease-in-out, opacity 0.3s ease-in-out, margin-top 0.3s ease;
}

/*display: block diganti dengan max-height: 200px dan opacity: 1*/
header nav.nav-open {
  max-height: 200px;
  opacity: 1;
  margin-top: 1rem;
}
```

3. Perluas `initTableFilter` supaya pencarian bisa dibatasi ke satu kolom saja (misalnya hanya kolom "Judul"), bukan mencari di seluruh teks baris — petunjuk: gunakan `row.querySelector("td")` seperti pola yang sudah dipakai di bab 5 §5.4, alih-alih `row.textContent`.

Jawaban

Mengubah pencarian di `app.js` agar hanya mencocokkan teks pada kolom pertama (`row.querySelector("td")`) alih-alih seluruh isi baris (`row.textContent`)

```
input.addEventListener("keyup", function () {
  const keyword = input.value.toLowerCase();
  const rows = table.querySelectorAll("tbody tr");
  rows.forEach(function (row) {
    //row.textContent menjadi row.querySelector("td").//
    const kolomPertama = row.querySelector("td");
    const teks = kolomPertama ? kolomPertama.textContent.toLowerCase() : "";
    row.style.display = teks.includes(keyword) ? "" : "none";
  });
});
```

Uji coba

Buka halaman daftar, ketik judul buku di kolom pencarian, lalu pastikan baris tabel berhasil menyaring data sesuai kata kunci tersebut.



4. Tambah counter jumlah baris tersisa setelah difilter atau dihapus — tampilkan misalnya "Menampilkan 3 dari 5 buku" di atas tabel, diperbarui setiap kali `initTableFilter` atau `initHapusConfirm` berjalan.

Jawaban

Tambah counter jumlah baris tersisa setelah difilter atau dihapus

- Penambahan elemen HTML pada `buku/list.html` dan `anggota/list.html`

```
</div>
<p id="table-counter" style="margin-bottom: 0.75rem; font-weight: 600; color: #55677a;"></p>
<div class="table-responsive">
```

- Pembuatan fungsi updateTableCounter()

```
function updateTableCounter() {
    const counterEl = document.getElementById("table-counter");
    const table = document.querySelector(".table-responsive table");
    if (!counterEl || !table) return;

    const totalRows = table.querySelectorAll("tbody tr");
    let visibleRowCount = 0;

    totalRows.forEach(function (row) {
        if (row.style.display !== "none") {
            visibleRowCount++;
        }
    });

    counterEl.textContent = "Menampilkan " + visibleRowCount + " dari " + totalRows.length + " data";
}
```

- Pemanggilan updateTableCounter() pada fungsi konfirmasi hapus (initHapusConfirm):

```
const yakin = confirm("Yakin ingin menghapus \"" + row.innerHTML + "\"?");
if (yakin && row) {
    row.remove();
    updateTableCounter(); // Perbarui counter
}
```

- Pemanggilan updateTableCounter() pada fungsi filter pencarian (initTableFilter):

```
const teks = kolomPertama ? kolomPertama.textContent.toLowerCase() : "";
row.style.display = teks.includes(keyword) ? "" : "none";
});
updateTableCounter(); // Perbarui counter setelah difilter
});
```

- Pemanggilan updateTableCounter() saat halaman pertama kali dimuat (DOMContentLoaded):

```
document.addEventListener("DOMContentLoaded", function () {
    initNavToggle();
    initHapusConfirm();
    initTableFilter();
    initValidasiForm();
    updateTableCounter(); // Jalankan counter saat halaman pertama kali dimuat
});
```

Uji coba

Buka halaman daftar, amati teks counter yang menampilkan total data awal, ketik kata kunci pencarian untuk menyaring baris, lalu lakukan penghapusan data untuk memastikan angka counter pada teks otomatis berkurang secara akurat.



teks counter berhasil memperbarui jumlah baris secara dinamis saat proses filter pencarian maupun penghapusan data baris tabel dilakukan

5. Refactor validasi — coba ubah `initValidasiForm` supaya nama field yang wajib divalidasi diambil dari sebuah array/daftar, alih-alih menulis blok if terpisah untuk tiap field satu-satu (petunjuk: pikirkan pola perulangan `forEach` yang sudah dipakai di bab 5 dan bab 6).

Jawaban

Ubah fungsi `initValidasiForm()` di file `assets/js/app.js` menggunakan pendekatan perulangan `forEach` dengan array konfigurasi rules/aturan validasi:

```
// ===== Refactor Validasi Form =====
function initValidasiForm() {
  const form = document.getElementById("form-tambah");
  if (!form) return;
  form.addEventListener("submit", function (e) {
    let valid = true;
    // Array konfigurasi aturan validasi
    const rules = [
      {
        selector: "[name='judul'], [name='nama']",
        test: (val) => val.trim() !== "",
        message: "Field ini wajib diisi."
      },
      {
        selector: "[name='pengarang']",
        test: (val) => val.trim() !== "",
        message: "Pengarang wajib diisi."
      },
      {
        selector: "[name='tahun']",
        test: (val) => {
          const nilai = parseInt(val, 10);
          return !isNaN(nilai) && nilai >= 1980 && nilai <= 2026;
        },
        message: "Tahun harus di antara 1980-2026."
      },
      {
        selector: "[name='isbn']",
        test: (val) => {
          if (val.trim() === "") return true; // ISBN opsional jika kosong
          return /^[0-9-]+$/i.test(val.trim());
        },
        message: "ISBN hanya boleh berisi angka dan tanda hubung (-)."
      }
    ];

    rules.forEach((rule) => {
      const element = document.querySelector(rule.selector);
      if (!element) return;
      const value = element.value;
      if (!rule.test(value)) {
        valid = false;
        alert(rule.message);
      }
    });

    if (!valid) e.preventDefault();
  });
}
```

```

        selector: "[name='stok']",
        test: (val) => {
            const nilai = parseInt(val, 10);
            return !isNaN(nilai) && nilai >= 0;
        },
        message: "Stok tidak boleh negatif."
    }
};

// Eksekusi perulangan validasi untuk setiap elemen
rules.forEach(function (rule) {
    const input = form.querySelector(rule.selector);
    if (input) {
        if (!rule.test(input.value)) {
            tampilkanError(input, rule.message);
            valid = false;
        } else {
            hapusError(input);
        }
    }
});

if (!valid) {
    e.preventDefault();
}
});
});

document.addEventListener("DOMContentLoaded", function () {
    initNavToggle();
    initHapusConfirm();
    initTableFilter();
    initValidasiForm();
    updateTableCounter(); // Jalankan counter saat halaman pertama kali dimuat
});

```

Uji coba
Mengisi form

Tambah Buku

Judul

Pengarang

Tahun Terbit

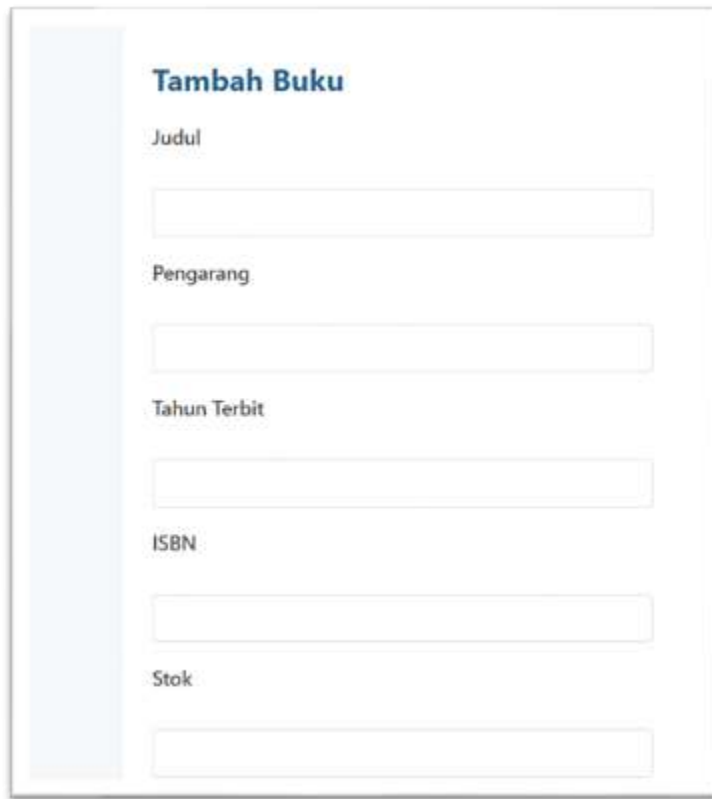
ISBN

Stok

Kategori

Fiksi

Klik simpan



Tambah Buku

Judul

Pengarang

Tahun Terbit

ISBN

Stok

Dan berhasil gk ada kata eror

Memasukkan data tidak valid misal isbn dan tahun yang tidak valid

Ketika memasukkan tahun yang gk valid muncul eror



Tahun Terbit

19

ISBN

afds

Nilai harus lebih dari atau sama dengan 1900.

Ketika memasukkan isbn yang gk valid akan eror



ISBN

afds

ISBN hanya boleh berisi angka dan tanda hubung (-).

Pengujian refaktor validasi form menggunakan array konfigurasi dan perulangan `forEach` berhasil dijalankan, di mana sistem mampu mengecek seluruh aturan input secara dinamis dan menampilkan pesan peringatan error dengan tepat apabila data yang dimasukkan tidak valid.

Jobshet 6

1. `fetch()` selalu asinkron — hasilnya berupa Promise, butuh `await` (di dalam `async function`) untuk mendapatkan data sungguhan (bab 1).

jawab

Menambahkan tombol muat ulang di samping pencarian

```
<label for="search-input">Cari Judul Buku</label>
<input type="text" id="search-input" placeholder="Ketik judul buku...">
<!-- tombol muat ulang -->
<button type="button" id="btn-reload" style="margin-left: 0.5rem; padding: 0.5rem 1rem; cursor: pointer;">Muat Ulang
```

Tambahkan event listener pada tombol `#btn-reload` di dalam berkas `buku.js` agar memanggil fungsi `muatDaftarBuku()` saat diklik:

```
if (!tbody) return;

if (loading) loading.style.display = "block";
tbody.innerHTML = ""; // Mengosongkan tbody sebelum me-render ulang

try {

  document.addEventListener("DOMContentLoaded", function () {
    muatDaftarBuku();

    // Event listener untuk tombol Muat Ulang
    const btnReload = document.getElementById("btn-reload");
    if (btnReload) {
      btnReload.addEventListener("click", muatDaftarBuku);
    }
  });
};
```

Telah ditambahkan tombol "Muat Ulang" pada halaman Daftar Buku (`buku/list.html`) yang terhubung dengan event listener `click` pada skrip `buku.js`. Ketika diklik, tombol ini mengeksekusi kembali fungsi asinkron `muatDaftarBuku()`. Karena fungsi tersebut secara otomatis mengosongkan elemen `<tbody>` (`tbody.innerHTML = ""`) sebelum melakukan pengambilan data ulang (`fetch`), maka pemanggilan berkali-kali aman dilakukan tanpa menimbulkan penumpukan (duplikasi) data pada tabel.

Hasil uji coba

- pengujian tombol muat ulang
Klik tombol "Muat Ulang" pada halaman Daftar Buku.



Judul	Pengarang	Tahun	Stok	Aksi
Laskar Pelangi	Andrea Hirata	2005	4	<button>Edit</button> <button>Hapus</button>
Bumi Manusia	Pramoedya Ananta Toer	1980	2	<button>Edit</button> <button>Hapus</button>
Negeri 5 Menara	Ahmad Fuadi	2009	0	<button>Edit</button> <button>Hapus</button>



Indikator memuat data (loading-indicator) tampil sejenak, tabel dikosongkan, lalu data buku dari buku.json dimuat ulang tanpa ada baris data ganda.

- Pengujian hapus data lalu muat ulang
Hapus salah satu baris buku menggunakan tombol hapus lalu klik tombol muat ulang



Klik ok



Klik muat ulang



Baris yang terhapus pada DOM sementara akan kembali muncul karena data diambil ulang secara segar (fresh fetch) dari berkas buku.json.

2. Satukan buku.js dan anggota.js jadi satu fungsi generik yang menerima nama file JSON dan daftar nama kunci sebagai parameter — latihan langsung untuk pertanyaan yang diajukan di bab 5 §5.4.

Membuat file baru yang bernama fetch-data.js dan menghapus file buku.js dan anggota.js

Fungsi generik ini menerima parameter urlJson (path file JSON) dan keys (array nama kunci/properti dari objek JSON yang ingin ditampilkan di kolom tabel).

```
* mengambil dan menampilkan data json ke tabel
* @param {string} urlJson - Path menuju file json
* @param {Array<string>} keys - array nama kunci objek yang ingin ditampilkan */

async function muatDaftarData(urlJson, keys) {
  const tbody = document.querySelector(".table-responsive table tbody");
  const loading = document.getElementById("loading-indicator");
  if (!tbody) return;

  if (loading) loading.style.display = "block";
  tbody.innerHTML = "";

  try {
    await new Promise((resolve) => setTimeout(resolve, 600));

    const res = await fetch(urlJson);
    if (!res.ok) {
      throw new Error("Gagal mengambil data (status " + res.status + ")");
    }
    const dataList = await res.json();

    dataList.forEach(function (item) {
      const tr = document.createElement("tr");
      let rowHtml = "";
```

Gantikan script buku.js atau anggota.js dengan memanggil fetch-data.js lalu jalankan fungsi muatDaftarData sesuai parameter datanya:

Pada buku/list.html

```
<p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 6</p>
</footer>
<script src="../../assets/js/app.js"></script>
<script src="../../assets/js/fetch-data.js"></script>
<script>
  const keysBuku = ["judul", "pengarang", "tahun", "stok"];
  const fetchBuku = () => muatDaftarData("../data/buku.json", keysBuku);
  document.addEventListener("DOMContentLoaded", fetchBuku);
  const btnReload = document.getElementById("btn-reload");
  if (btnReload) btnReload.addEventListener("click", fetchBuku);
```

Pada anggota/list.html

```
<p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 6</p>
</footer>
<script src="../../assets/js/app.js"></script>
<script src="../../assets/js/fetch-data.js"></script>
<script>
const keysAnggota = ["no_anggota", "nama", "alamat", "no_hp"];
document.addEventListener("DOMContentLoaded", function () {
  muatDaftarData("../data/anggota.json", keysAnggota);
});
</script>
```

Penggabungan fungsi muatDaftarBuku() dan muatDaftarAnggota() menjadi fungsi generik muatDaftarData(urlJson, keys) berhasil menerapkan prinsip Don't Repeat Yourself (DRY). Fungsi ini menerima parameter URL file JSON dan daftar nama kunci (keys) untuk merender baris dan kolom tabel secara dinamis, sehingga modul JavaScript menjadi lebih fleksibel, efisien, dan reusable untuk berbagai struktur data di SIMPUS-Mini."

Uji coba

- Uji coba halaman buku
Buka halaman Daftar Buku lewat Live Server.



Judul	Pengarang	Tahun	Stok	Aksi
Laskar Pelangi	Andrea Hirata	2005	4	<button>Edit</button> <button>Hapus</button>
Bumi Manusia	Pramoedya Ananta Toer	1980	2	<button>Edit</button> <button>Hapus</button>
Negeri 5 Menara	Ahmad Fuadi	2009	0	<button>Edit</button> <button>Hapus</button>

Hasil: Indikator "Memuat data..." tampil selama 600ms, lalu tabel terisi 4 kolom data (judul, pengarang, tahun, stok) dan 1 kolom aksi. Klik tombol Muat Ulang untuk memastikan re-fetch berhasil.

- Uji coba halaman anggota
Buka halaman Daftar Anggota.

No. Anggota	Nama	Alamat	No. HP	Aksi
A001	Siti Aminah	Malang	0812xxxx	<button>Edit</button> <button>Hapus</button>
A002	Budi Santoso	Batu	0813xxxx	<button>Edit</button> <button>Hapus</button>
A003	Dewi Lestari	Malang	0814xxxx	<button>Edit</button> <button>Hapus</button>
A004	Rizky Firmansyah	Lawang	0815xxxx	<button>Edit</button> <button>Hapus</button>

Hasil: Tabel terisi 4 kolom data (no_anggota, nama, alamat, no_hp) dan 1 kolom aksi secara dinamis tanpa ada error di Console browser.

- Uji coba fitur global
Ketik nama pada search box dan klik tombol Hapus pada salah satu baris.

Judul	Pengarang	Tahun	Stok	Aksi
Negeri 5 Menara	Ahmad Fuadi	2009	0	<button>Edit</button> <button>Hapus</button>

Hasil: Pencarian realtime dan penghapusan baris tetap berjalan normal karena event delegation pada app.js tetap terhubung.

3. Tambah kolom baru di data/buku.json (misalnya "kategori"), lalu tampilkan di tabel dengan menambah satu `<th>` di HTML dan satu `<td>` di `tr.innerHTML` pada buku.js.

Jawab

Update file data/buku.json

Tambahkan properti "kategori" pada setiap objek data buku:

```

1  {
2    { "judul": "Laskar Pelangi", "pengarang": "Andrea Hirata", "tahun": 2005, "stok": 4, "kategori": "Novel" },
3    { "judul": "Bumi Manusia", "pengarang": "Pramoedya Ananta Toer", "tahun": 1980, "stok": 2, "kategori": "Sastra" },
4    { "judul": "Negeri 5 Menara", "pengarang": "Ahmad Fuadi", "tahun": 2009, "stok": 0, "kategori": "Novel" },
5    { "judul": "Filosofi Teras", "pengarang": "Henry Manampiring", "tahun": 2018, "stok": 5, "kategori": "Pengembangan Diri" },
6    { "judul": "Ronggeng Dukuh Paruk", "pengarang": "Ahmad Tohari", "tahun": 1982, "stok": 1, "kategori": "Sastra" },
7    { "judul": "Cantik Itu Luka", "pengarang": "Eka Kurniawan", "tahun": 2002, "stok": 3, "kategori": "Novel" },
8    { "judul": "Pulang", "pengarang": "Tere Liye", "tahun": 2015, "stok": 2, "kategori": "Action" },
9    { "judul": "Sang Pemimpi", "pengarang": "Andrea Hirata", "tahun": 2006, "stok": 6, "kategori": "Novel" },
10   { "judul": "Perahu Kertas", "pengarang": "Dee Lestari", "tahun": 2009, "stok": 0, "kategori": "Romance" },
11   { "judul": "Gadis Kretek", "pengarang": "Ratih Kumala", "tahun": 2012, "stok": 4, "kategori": "Fiksi Sejarah" }
12 }

```

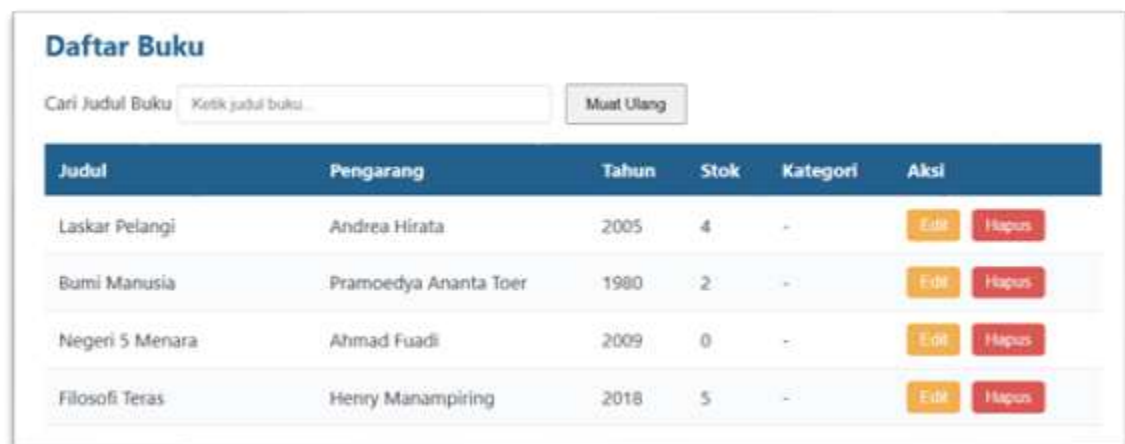
Update file buku/list.html

Tambahkan elemen `<th>Kategori</th>` pada tabel HTML, dan masukkan "kategori" ke dalam daftar `keysBuku`:

```
<th>Tahun</th>
<th>Stok</th>
<th>Kategori</th>
<th>Aksi</th>

<script>
const keysBuku = ["judul", "pengarang", "tahun", "stok", "kategori"];
const fetchBuku = () => muatDaftarData("../data/buku.json", keysBuku);
```

Hasil



The screenshot shows a web application titled "Daftar Buku". It features a search bar labeled "Cari Judul Buku" with a placeholder "Ketik judul buku..." and a "Muat Ulang" button. Below the search bar is a table with the following columns: Judul, Pengarang, Tahun, Stok, Kategori, and Aksi. The table contains four rows of book data, each with "Edit" and "Hapus" buttons in the Aksi column.

Judul	Pengarang	Tahun	Stok	Kategori	Aksi
Laskar Pelangi	Andrea Hirata	2005	4	-	Edit Hapus
Bumi Manusia	Pramoedya Ananta Toer	1980	2	-	Edit Hapus
Negeri 5 Menara	Ahmad Fuadi	2009	0	-	Edit Hapus
Filosofi Teras	Henry Manampiring	2018	5	-	Edit Hapus

Penambahan kolom Kategori pada `data/buku.json` dan elemen `<th>` pada `buku/list.html` dapat langsung ter-render secara otomatis tanpa mengubah logika JavaScript. Hal ini karena fungsi generik `muatDaftarData()` secara dinamis membuat `<td>` berdasarkan array `keysBuku` yang dikirimkan sebagai parameter."

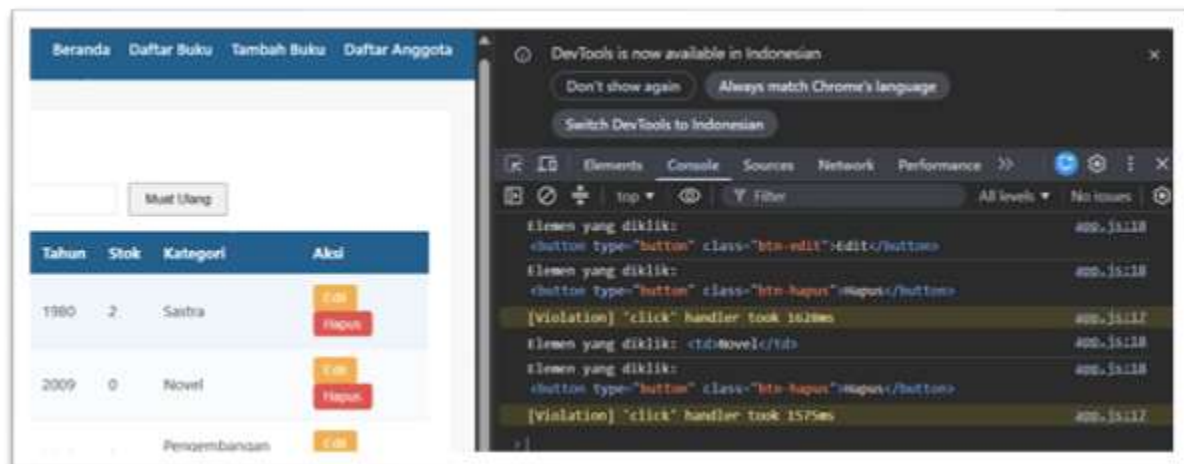
4. Uji delegasi event lebih jauh — tambahkan `console.log(e.target)` di awal fungsi `initHapusConfirm` (bab 6), buka Console, lalu klik berbagai tempat di halaman (bukan cuma tombol Hapus) untuk melihat sendiri bagaimana document menerima semua event klik di halaman, dan bagaimana `e.target.closest(...)` menyaring hanya yang relevan.

Jawab

- Update Fungsi di `assets/js/app.js`
Tambahkan `console.log(e.target)` di baris pertama di dalam event listener.

```
document.addEventListener("click", function (e) {
  console.log("Elemen yang diklik:", e.target);
  // ... (target.closest(...)) ...
```

- Uji coba



Hasil uji coba menggunakan `console.log(e.target)` membuktikan bahwa event listener pada tingkat document menerima semua event klik di halaman (event bubbling). Namun, dengan metode `e.target.closest('.btn-hapus')`, kita dapat memfilter event tersebut secara akurat sehingga fungsi eksekusi hanya berjalan saat tombol Hapus yang diklik, tanpa perlu menempelkan event listener satu per satu pada setiap baris tabel secara manual."

5. Ganti delay simulasi di bab 4 §4.4 dari 600 menjadi 3000 (3 detik), amati loading indicator jadi jauh lebih terlihat — ini juga cara yang baik untuk merasakan pentingnya loading indicator pada koneksi yang lambat.

Jawab

Ganti delay dari 600ms menjadi 3000ms (3 detik)

```
try {
  // Ganti delay
  await new Promise(resolve) => setTimeout(resolve, 3000);
}
```



Mengubah delay simulasi `setTimeout` menjadi 3000 ms membuktikan peran krusial indikator loading dalam memberikan umpan balik visual (user feedback) kepada pengguna. Pada kondisi jaringan lambat atau latency tinggi, indikator ini mengonfirmasi bahwa proses pengambilan data (asynchronous fetch) sedang berjalan sehingga pengguna tidak menganggap aplikasi macet atau error.